COMP9414 artificial intelligence

Tutorial topic: knowledge representation with ai9414

UNSW Sydney

1 Introduction

This tutorial practises the main ideas from the knowledge representation lecture: propositional symbols, truth assignments, entailment, DPLL SAT solving, and resolution refutation. The aim is to be able to answer questions such as:

- What does a knowledge base claim about the possible worlds?
- What does it mean for a query to be entailed?
- Why does DPLL search for a satisfying assignment?
- Why does resolution add the negated query and derive the empty clause?

The tutorial uses two local `ai9414` demos:

- `logic-dpll`: visual DPLL over CNF clauses and entailment examples.
- `logic-resolution`: visual resolution/refutation proofs.

Notation reminder. In this document, $B_{1,1}$ means "there is a breeze in square $[1,1]$", while $P_{1,2}$ means "there is a pit in square $[1,2]$". The symbols are just propositional variables; DPLL and resolution do not know about squares unless the knowledge base encodes those relationships.

2 Installation and setup

2.1 Create a Python environment

The package requires Python 3.11 or newer. On macOS or Linux, create and activate a virtual environment as follows:

```
mkdir ai9414-logic-tutorial
cd ai9414-logic-tutorial
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install "ai9414>=0.1.5"
```

On Windows PowerShell, the activation command is usually:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install "ai9414>=0.1.5"
```

This tutorial uses `ai9414` version `0.1.5` or newer, which includes both the `logic-dpll` and `logic-resolution` demos.

Check that the command-line tool is available:

```
ai9414 list
python -m pip show ai9414
```

If the command is not on your `PATH`, use the module entry point:

```
python -m ai9414 list
```

2.2 Run the logic demos

Each command starts a local web app and opens a browser window. Leave the command running while you use the demo, then stop it with `Ctrl-C`.

```
ai9414 demo logic-dpll --example unit_chain
ai9414 demo logic-dpll --example unsat_branching
ai9414 demo logic-dpll --example wumpus_no_breeze
ai9414 demo logic-resolution --example chain_refutation
ai9414 demo logic-resolution --example wumpus_no_breeze
```

To see the available examples:

```
ai9414 list --examples logic-dpll
ai9414 list --examples logic-resolution
```

The current DPLL examples are:

- `simple_sat`, `tiny_contradiction`, `unit_chain`
- `unsat_branching`, `constraint_model`, `entailment_chain`
- `wumpus_no_breeze`, `wumpus_forced_pit`

The current resolution examples are:

- `modus_ponens_refutation`, `chain_refutation`
- `wumpus_no_breeze`, `wumpus_forced_pit`

3 Propositional sentences and models

A propositional symbol is a statement that can be true or false. A model is a complete truth assignment for the symbols under discussion.

Symbol	Intended reading	One possible value
K	the robot has the keycard	true
O	the office door is open	false
$B_{1,1}$	there is a breeze at $[1, 1]$	false
$P_{1,2}$	there is a pit at $[1, 2]$	true

Logical connectives build compound sentences:

Formula	Read as
$\neg P$	not P
$P \wedge Q$	P and Q
$P \vee Q$	P or Q
$P \to Q$	if P, then Q
$P \leftrightarrow Q$	P if and only if Q

3.1 Task: evaluate a formula

Use the assignment

$$K = T, \quad O = F, \quad B_{1,1} = F, \quad P_{1,2} = T.$$

For each sentence, write whether it is true or false and give one sentence of reasoning.

1. $K \to O$
2. $\neg B_{1,1}$
3. $P_{1,2} \to B_{1,1}$
4. $(K \vee O) \wedge \neg B_{1,1}$
5. $B_{1,1} \leftrightarrow P_{1,2}$

Useful shortcut. An implication $P \to Q$ is false only when P is true and Q is false. If the right-hand side is true, the implication is true. If the left-hand side is false, the implication is also true.

4 Entailment and the Wumpus rule

Entailment is a semantic relationship:

$$KB \models \alpha.$$

Read this as "the knowledge base entails α". It means every model that makes KB true also makes α true.

In the Wumpus world, one small rule is:

$$B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1}).$$

This says square $[1, 1]$ is breezy exactly when one of its adjacent squares contains a pit.

Suppose the agent observes no breeze:

$$\neg B_{1,1}.$$

Then the knowledge base should entail:

$$\neg P_{1,2} \qquad \text{and} \qquad \neg P_{2,1}.$$

The observation is about B; the query is about P. Logic connects them through the rule.

4.1 Task: DPLL Wumpus no-breeze example

Run:

```
ai9414 demo logic-dpll --example wumpus_no_breeze
```

Step through the replay and answer:

1. What is the query shown by the demo?
2. What assumption is added as the negated query?
3. Which clause forces $B_{1,1}$ if $P_{1,2}$ is assumed?
4. Where does the contradiction appear?
5. In one sentence, explain why the contradiction proves $KB \models \neg P_{1,2}$.

5 DPLL as SAT search

DPLL is a complete backtracking procedure for propositional satisfiability. Given a CNF formula, it tries to build a truth assignment that makes every clause true.

$$(A) \wedge (\neg A \vee B) \wedge (\neg B \vee C)$$

A raw truth table checks every complete row. DPLL usually does less work because it works with partial assignments.

Situation under partial assignment	DPLL action
Every clause is satisfied	return SAT and the assignment
Some clause is contradicted	abandon this branch
A clause has one unresolved literal	force that literal to make the clause true
No forced move exists	choose a symbol and branch true/false

5.1 Task: unit propagation

Run:

```
ai9414 demo logic-dpll --example unit_chain
```

The example uses:

$$A, \quad \neg A \vee B, \quad \neg B \vee C.$$

Fill in the table:

Step	New assignment	Why forced?
1		
2		
3		

5.2 Task: branching and failure

Run:

```
ai9414 demo logic-dpll --example unsat_branching
```

This example is:

$$(A \vee B) \wedge (\neg A \vee B) \wedge (A \vee \neg B) \wedge (\neg A \vee \neg B).$$

Answer:

1. Why is there no immediate unit clause at the start?
2. Which variable does the demo branch on first?
3. Explain why the first branch fails.
4. Explain why the second branch also fails.
5. What is the final SAT/UNSAT result?

6 Resolution refutation

Resolution is a proof method over clauses. A clause is a disjunction of literals, for example:

$$\neg A \vee B.$$

The resolution rule cancels one complementary pair:

$$\frac{A \vee X \qquad \neg A \vee Y}{X \vee Y}.$$

To prove entailment by refutation, add the negated query and derive the empty clause:

$$KB \models \alpha \quad \text{by showing} \quad CNF(KB \wedge \neg\alpha) \vdash_{Res} \square.$$

The empty clause $\square$ is pronounced "box" or "the empty clause". It means contradiction.

6.1 Task: chain refutation

Run:

```
ai9414 demo logic-resolution --example chain_refutation
```

The knowledge base is:

$$A \to B, \quad B \to C, \quad A.$$

The query is C. The refutation clause set is:

$$\neg A \vee B, \quad \neg B \vee C, \quad A, \quad \neg C.$$

Fill in the missing derived clauses:

Step	Clause	Reason
1	$\neg A \vee B$	from $A \to B$
2	$\neg B \vee C$	from $B \to C$
3	A	fact
4	$\neg C$	negated query
5		resolve 1 and 3
6		resolve 2 and 5
7		resolve 4 and 6

6.2 Task: Wumpus resolution

Run:

```
ai9414 demo logic-resolution --example wumpus_no_breeze
```

The query is $\neg P_{1,2}$. The refutation adds $P_{1,2}$.
Answer:

1. Which clause comes from the observation?
2. Which clause is the negated query?
3. Which two clauses resolve to derive $B_{1,1}$?
4. Which two clauses then derive $\square$?
5. Why is the proof about $P_{1,2}$ even though the contradiction is about $B_{1,1}$?

7 Comparing proof methods

The same entailment question can be attacked in several ways.

Method	What it searches/manipulates	Evidence of success
Model checking	complete truth assignments	every KB model satisfies the query
DPLL	partial truth assignments	$KB \wedge \neg query$ is UNSAT
Resolution	clauses and derived clauses	derivation reaches $\square$
Forward chaining	definite-clause rules	query fact is derived

7.1 Task: method comparison

For the query

$$\{A \to B,\ B \to C,\ A\} \models C,$$

write one sentence for each method:

1. What would model checking inspect?
2. What would DPLL try to satisfy?
3. What would resolution derive?
4. What would forward chaining add to the known facts?

8 Optional extension: live Python DPLL

The DPLL demo can call a local Python solver. In the browser, use the live Python mode and download the Python stub. The stub starts a small local server and calls your function:

```
def solve_dpll(problem, options):
    ...
```

A minimal valid result has this shape:

```
{
    "algorithm": "dpll",
    "mode": "sat",              # or "entailment"
    "status": "satisfiable",    # or "unsatisfiable", "entailed", ...
    "assignment": {"A": True},
    "trace": [
        {
            "step": 0,
            "action": "start",
            "node_id": "n0",
            "parent_id": None,
            "variable": None,
            "value": None,
            "reason": None,
            "clause_index": None,
            "assignment": {},
        }
    ],
}
```

You do not need a sophisticated SAT solver for this tutorial. A useful first implementation is:

1. detect satisfied, contradicted, and unresolved clauses;
2. repeatedly apply unit clauses;
3. if nothing is forced, branch on the alphabetically first unassigned symbol;
4. backtrack when a clause is contradicted.

9 Your mission

Complete the following tasks during the tutorial.

1. **Environment check.** Install `ai9414`, run `ai9414 list`, and launch `ai9414 demo logic-dpll --example unit_chain`. Record the example title and the final SAT/UNSAT result.
2. **Formula evaluation.** Complete the five truth-value questions in the "Propositional Sentences and Models" section.
3. **Wumpus entailment.** Run the `wumpus_no_breeze` DPLL example and answer the five Wumpus questions. Be explicit about the difference between the observation $\neg B_{1,1}$ and the query $\neg P_{1,2}$.
4. **DPLL trace.** Complete the unit-propagation table and the unsatisfiable-branching questions.
5. **Resolution trace.** Run `logic-resolution --example chain_refutation` and fill in the missing clauses in the proof table.
6. **Resolution Wumpus.** Run `logic-resolution --example wumpus_no_breeze` and identify the two resolution steps that produce $B_{1,1}$ and then $\square$.

7. **Compare methods.** Complete the method-comparison task for $\{A \to B,\ B \to C,\ A\} \models C$.

8. **Reflection.** In three or four sentences, explain why DPLL is called model search and resolution is called proof search.

10 Task checklist

If this was a test or exam, you would be asked to submit:

- the environment check result;
- truth values and explanations for the five formulae;
- Wumpus DPLL answers distinguishing observation, query, and negated query;
- the completed unit-propagation and branching notes;
- the completed resolution chain table;
- the Wumpus resolution explanation;
- the short comparison of model checking, DPLL, resolution, and forward chaining;
- the final reflection paragraph.

A Logic symbol quick reference

Symbol	Read as	Role
$\models$	entails	semantic relationship over models
$\vdash$	proves/derives	syntactic derivation inside a proof system
$\square$	box / empty clause	contradiction in resolution
$\leftrightarrow$	if and only if	connective inside a formula
$\equiv$	logically equivalent to	meta-level rewrite/equivalence claim
$=$	equals	identity or equality of values/objects

B CNF rewrite reminders

Resolution and DPLL both operate over CNF clauses. Useful rewrites include:

$$
\begin{aligned}
P \to Q &\equiv \neg P \vee Q \\
P \leftrightarrow Q &\equiv (P \to Q) \wedge (Q \to P) \\
\neg(P \wedge Q) &\equiv \neg P \vee \neg Q \\
\neg(P \vee Q) &\equiv \neg P \wedge \neg Q
\end{aligned}
$$

Example:

$$A \to B \quad \text{becomes} \quad \neg A \vee B.$$

C Demo command quick reference

```
ai9414 list
ai9414 list --examples logic-dpll
ai9414 list --examples logic-resolution

ai9414 demo logic-dpll --example simple_sat
ai9414 demo logic-dpll --example unit_chain
ai9414 demo logic-dpll --example unsat_branching
ai9414 demo logic-dpll --example wumpus_no_breeze
ai9414 demo logic-dpll --example wumpus_forced_pit

ai9414 demo logic-resolution --example chain_refutation
ai9414 demo logic-resolution --example wumpus_no_breeze
ai9414 demo logic-resolution --example wumpus_forced_pit
```

References

[1] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.

[2] Stuart Russell and Peter Norvig. *AIMA Chapter 7: Logical Agents* slides. `https://aima.eecs.berkeley.edu/slides-pdf/chapter07.pdf`.

[3] UC Berkeley CS188. *Inference in Propositional Logic* lecture material.

[4] Stanford CS221. *Logic and Resolution* lecture material.

[5] Oliver Obst. *ai9414: Interactive teaching demos for artificial intelligence.* `https://pypi.org/project/ai9414/`.